

小结

在 C++ 中，内存是通过 `new` 表达式分配，通过 `delete` 表达式释放的。标准库还定义了一个 `allocator` 类来分配动态内存块。

分配动态内存的程序应负责释放它所分配的内存。内存的正确释放是非常容易出错的地方：要么内存永远不会被释放，要么在仍有指针引用它时就被释放了。新的标准库定义了智能指针类型——`shared_ptr`、`unique_ptr` 和 `weak_ptr`，可令动态内存管理更为安全。对于一块内存，当没有任何用户使用它时，智能指针会自动释放它。现代 C++ 程序应尽可能使用智能指针。

术语表

allocator 标准库类，用来分配未构造的内存。

空悬指针 (dangling pointer) 一个指针，指向曾经保存一个对象但现在已释放的内存。众所周知，空悬指针引起的程序错误非常难以调试。

delete 释放 `new` 分配的内存。`delete p` 释放对象，`delete []p` 释放 `p` 指向的数组。`p` 可以为空，或者指向 `new` 分配的内存。

释放器 (deleter) 传递给智能指针的函数，用来代替 `delete` 释放指针绑定的对象。

析构函数 (destructor) 特殊的成员函数，负责在对象离开作用域或被释放时完成清理工作。

动态分配的 (dynamically allocated) 在自由空间中分配的对象。在自由空间中分配的对象直到被显式释放或程序结束才会销毁。

自由空间 (free store) 程序可用的内存池，保存动态分配的对象。

堆 (heap) 自由空间的同义词。

new 从自由空间分配内存。`new T` 分配并构造一个类型为 `T` 的对象，并返回一个指向该对象的指针。如果 `T` 是一个数组类型，`new` 返回一个指向数组首元素的指针。类似的，`new [n] T` 分配 `n` 个类型为 `T` 的对象，并返回指向数组首元素的指针。

默认情况下，分配的对象进行默认初始化。我们也可以提供可选的初始化器。

定位 new (placement new) 一种 `new` 表达式形式，接受一些额外的参数，在 `new` 关键字后面的括号中给出。例如，`new (nothrow) int` 告诉 `new` 不要抛出异常。

引用计数 (reference count) 一个计数器，记录有多少用户共享一个对象。智能指针用它来判断什么时候释放所指向的对象是安全的。

shared_ptr 提供所有权共享的智能指针：对共享对象来说，当最后一个指向它的 `shared_ptr` 被销毁时会被释放。

智能指针 (smart pointer) 标准库类型，行为类似指针，但可以检查什么时候使用指针是安全的。智能指针类型负责在恰当的时候释放内存。

unique_ptr 提供独享所有权的智能指针：当 `unique_ptr` 被销毁时，它指向的对象被释放。`unique_ptr` 不能直接拷贝或赋值。

weak_ptr 一种智能指针，指向由 `shared_ptr` 管理的对象。在确定是否应释放对象时，`shared_ptr` 并不把 `weak_ptr` 统计在内。